ADITYA UNIVERSITY

SCHOOL OF ENGINEERING

Department of Data Science

CERTIFICATE

This is to certify that the Cornerstone project work entitled "**STUDENT COURSE REGISTRATION SYSTEM**" is submitted by

T KALPANA	**(24B11DS208)**
V YAMINI SRI	**(24B11DS221)**
K SUSHMA	**(24B11DS082)**
Y YASASWINI	**(24B11DS231)**

in partial fulfillment of the requirements for the award of the B.Tech. degree in Data Science for the academic year 2025-2026.

Project Guide
Dr. K Sreenu, Ph.D,
Assistant Professor,
Department of DS

Head of the Department
Dr. M. Rajababu, Ph.D,
Assistant Professor,
Department of DS

DECLARATION

We hereby declare that the Cornerstone project entitled "**STUDENT COURSE REGISTRATION SYSTEM**" is original and is submitted to **ADITYA UNIVERSITY**, Surampalem, in partial fulfillment of the requirements for the award of the **B.Tech. degree** in **Data Science**. We further declare that this project work has not been submitted, either in full or in part, for the award of any degree in this or any other institution. We also confirm that this work complies with the Institutional Academic Integrity and AI Usage Policy

Place: Surampalem

Date:

By

T KALPANA	(24B11DS208)
V YAMINI SRI	(24B11DS221)
K SUSHMA	(24B11DS082)
Y YASASWINI	(24B11DS231)

ACKNOWLEDGEMENT

It is with immense pleasure that we express our sincere gratitude to our Cornerstone Project Guide, **Dr. K Sreenu, Assistant Professor**, who has guided us throughout the project. His valuable support, encouragement, and guidance have greatly contributed to the successful completion of this work.

We would like to thank our Deputy Heads of the Department, **Dr. A. Lakshmana Rao, Associate Professor and Mrs. S. Neelima, Assistant Professor**, Department of DS, for their support and valuable suggestions.

Our deepest thanks to our Head of the Department **Dr. M. Rajababu, Associate Professor,** for his inspiration and for providing all the necessary facilities and resources required for this project.

We also extend our sincere thanks to **Dr. M.V. Rajesh, Associate Dean, School of Engineering** for their encouragement and support during the course of our project.

We would like to express our sincere gratitude to **Dr. G. Suresh, Registrar, Dr. A Ramesh, Pro Vice-Chancellor (Engineering & Sciences), Dr. S. Rama Sree, Pro Vice-Chancellor (Academics), Dr. M.B. Srinivas, Vice-Chancellor, Dr. M. Sreenivasa Reddy, Deputy Pro Chancellor and Management, Aditya University** for providing excellent infrastructure and state-of-the-art laboratories.

Finally, we would like to thank the **faculty members, lab technicians, non-teaching staff, and our friends** who have directly or indirectly supported us in completing this Cornerstone project successfully.

Table of contents

CONTENTS	PAGENO
ABSTRACT	i
LIST OF FIGURES	ii
LIST OF TABLES	iii
ACRONYMS	iv
1. INTRODUCTION	1
1.1 Brief Information about the Project	1
1.2 Motivation and Contribution of Project	1
1.3 Objectives of the Project	1
1.4 Scope of the Project	2
2. SYSTEM ANALYSIS	3
2.1 Functional Requirements	3
2.2 Non-Functional Requirements	4
2.3 Requirements Specification	4
2.3.1 Minimum Hardware Requirements	4
2.3.2 Software Requirements	4
3. TECHNOLOGY DESCRIPTION	5
3.1 Programming Language	5
3.1.1 Introduction to Java	5
3.2 Framework / Libraries	5
3.2.1 Introduction to Swing	5
3.3 Database Technology	6

CONTENTS **PAGENO**

3.3.1 JDBC (Java Database Connectivity) 6

3.3.2 Oracle Database 6

3.4 Tools & IDEs 7

3.4.1 Visual Studio Code 7

4. DATABASE DESIGN 8

4.1 Database Tables 8

4.2 Table Structure 8

4.3 Keys and Constraints 9

4.4 Table Relationships 10

5. SYSTEM DESIGN 11

5.1 System Architecture 11

5.2 Modules Description 12

6. SYSTEM IMPLEMENTATION 13

7. CONCLUSION 17

8. FUTURE ENHANCEMENTS 18

9. REFERENCES 19

APPENDIX (Sample Code) 20

APPENDIX (SQL Queries) 23

ABSTRACT

The Student Course Registration System (SCRS) is a desktop-based application designed to automate the process of student course enrolment in educational institutions. The system replaces traditional manual methods such as paper-based registration and spreadsheets with a structured digital solution. The application is developed using Core Java for implementing logic, Swing for graphical user interface design, JDBC for database connectivity, and Oracle Database for backend data storage. The system allows administrators to manage student details, course information, and course registrations efficiently. It also provides analytical features such as counting registrations for specific students and courses. The system improves data accuracy, reduces redundancy, and enhances overall efficiency in managing academic records.

LIST OF FIGURES

Figure No.	Figure Title	Page No.
1	Login Screen of Student Course Registration System	10
2	Main Menu Screen of Student Course Registration System	10
3	Student Management Screen of Student Course Registration System	10
4	Course Management Screen of Student Course Registration System	10
5	Course Registration Screen of Student Course Registration System	11
6	Student Subject Registration Count Screen of Student Course Registration System	11
7	Section Registration Count Screen of Student Course Registration System	11

LIST OF TABLES

Table no.	Table Title	Page no.
1	Student Relational Database Table	7
2	Course Relational Database Table	7
3	Registration Relational Database Table	7

ACRONYMS

- **SCRS :** Student Course Registration System
- **JDBC :** Java Database Connectivity
- **GUI :** Graphical User Interface
- **SQL :** Structured Query Language

1. INTRODUCTION

1.1 Brief Information about the Project

The Student Course Registration System (EMS) is a desktop-based application developed to manage student course enrolment efficiently within educational institutions. It provides a centralized platform to store, retrieve, update, and manage student, course, registration in a structured manner. The system replaces traditional manual record-keeping methods with a computerized solution, thereby improving data accuracy, reducing redundancy, and enhancing accessibility. The application is developed using Core Java for programming, Swing for designing the graphical user interface, JDBC for database connectivity, and Oracle Database for storing employee data. It is designed to be user-friendly and suitable for use in colleges and educational institutions for effective academic management.

1.2 Motivation and Contribution of Project

In many organizations, employee data is still maintained using manual registers or basic spreadsheet tools, which leads to inefficiencies such as data duplication, difficulty in managing large records, slow processing and increased chances of errors or data loss. These limitations motivated the development of the Student Course Registration System.

The key contributions of this project include:

- Providing a digital solution for managing student and course registration data
- Reducing manual effort and paperwork involved in the registration process
- Improving accuracy and consistency of academic records
- Enabling quick search and retrieval of student and course information
- Maintaining structured relationships between students and courses
- Enhancing data management and accessibility

This system simplifies the course registration process and improves overall efficiency in academic management.

1.3 Objectives of the Project

The main objectives of the Employee Management System are:

- To develop a user-friendly application for managing student course registration
- To store student and course information in a structured database
- To provide functionalities for adding, updating, and deleting student and course records

- To enable efficient course registration for students
- To allow quick searching of student and course details using ID or name
- To maintain proper relationships between students and courses

1.4 Scope of the Project

The Student Course Registration System can be used colleges and educational institutions to manage student course enrolment efficiently. It supports operations such as adding student and course details, updating existing records, deleting information, registering students for courses, searching for records using ID or name, and displaying all data in an organized manner. The system provides a simple and user-friendly interface for academic data in a centralized database.

2. SYSTEM ANALYSIS

2.1 Functional Requirements

Functional requirements define the core operations and functionalities that the Student Course Registration System (SCRS) must perform to manage student, course, and registration data efficiently. The Functional requirements of Student Course Registration System (SCRS)

- **Add Student Details**
  The system should allow the administrator to add new student records, including details such as student ID, name, contact information, and other relevant data. Proper validation should be applied to ensure data accuracy.
- **Update Student Records**
  The system should provide functionality to modify or update existing student information whenever required. This includes editing personal details such as name, email, or phone number.
- **Delete Student Information**
  The system should allow authorized users to delete student records from the database. A confirmation mechanism should be included to prevent accidental deletion.
- **Manage Course Details**
  The system should allow administrators to add, update, and delete course information, including course ID, course name, credits, and section details.
- **Course Registration**
  The system shall allow students to be registered for courses. It shall maintain proper relationships between students and courses and ensure valid registration without duplication.
- **Search Records**
  The system should support searching for students or courses using specific criteria such as ID or name. The search results should be accurate and fast.
- **Display Records**
  The system shall display all stored data such as student details, course details, and registration information in a structured and readable format.

2.2 Non-Functional Requirements

Non-functional requirements define the quality attributes and performance standards that the **Student Course Registration System** must satisfy.

- **Performance**
  The system should respond quickly to operations such as student registration, course management, and data retrieval without noticeable delays.
- **Security**
  Only authorized users should be able to access the system using secure login credentials. Proper authentication and access control mechanisms should be implemented to protect data.
- **Usability**
  The system should provide a simple, intuitive, and user-friendly interface so that users can perform tasks easily without requiring technical expertise.
- **Reliability**
  The system should operate consistently without failures and should handle errors gracefully without loss of data.
- **Scalability**
  The system should be capable of handling an increasing number of students, courses and registrations without degradation in performance.

2.3 Requirements Specification

This section specifies the hardware and software needed to run the system.

2.3.1 Minimum Hardware Requirements

- Processor: Intel i3 or above
- RAM: 4 GB (minimum)
- Hard Disk: 20 GB free space
- Monitor: Standard display
- Keyboard and Mouse

2.3.2 Software Requirements

- **Operating System:** Windows 7/10/11
- **Programming Language:** Java
- **IDE:** VS Code
- **Database:** Oracle Database
- **Connectivity:** JDBC

3. TECHNOLOGY DESCRIPTION

3.1 Programming Language

3.1.1 Introduction to Java

Java is a high-level, object-oriented programming language developed by Sun Microsystems. It is widely used for developing desktop, web, and enterprise applications due to its platform independence and robustness.

In the Student Course Registration System, Java is used to implement the core functionality of the application. It handles operations such as managing student details, course information, and course registration processes. Java provides features like object oriented programming, exception handling, and strong memory management, which help in building reliable and efficient applications.

One of the main advantages of Java is that it is platform-independent, meaning the program can run on any system that has a Java Virtual Machine (JVM). This makes it suitable for developing portable and scalable applications like the Student Course Registration System.

3.2 Framework / Libraries

3.2.1 Introduction to Swing

Swing is a part of Java that is used to create graphical user interfaces (GUI). It provides a set of components such as buttons, labels, text fields, tables, and frames to design user-friendly applications.

In this project, Swing is used to design the user interface of the Student Course Registration System. It allows users to interact with the system easily through windows such as:

- Login Window
- Main Menu Screen
- Student Management Screen
- Course Management Screen
- Course Registration Screen
- Display Records Screen
- Student Subject Registration Count Screen
- Section Registration Count Screen

Swing provides flexibility in designing layouts and improves the overall user experience. It makes the application visually interactive and easy to use.

3.3 Database Technology

The Student Course Registration System uses a relational database for storing and managing student, course, and registration information in a structured manner. The database ensures efficient data storage, retrieval, and manipulation, which is essential for maintaining accuracy and consistency in the system. In this project, Oracle Database is used as the backend, and JDBC is used for connecting the Java application to the database, enabling smooth communication between the application and the database..

3.3.1 JDBC (Java Database Connectivity)

JDBC is an API (Application Programming Interface) used in Java to connect and interact with databases. It allows Java applications to execute SQL queries and retrieve or update data in the database.

In the Student Course Registration System, JDBC is used to connect the Java application with the Oracle database. It performs operations such as:

- Establishing a connection to the database
- Executing SQL queries (INSERT, UPDATE, DELETE, SELECT)
- Retrieving data from the database
- Closing the database connection

JDBC acts as a bridge between the application and the database, ensuring smooth communication and data transfer. It helps in maintaining data consistency and enables efficient data handling.

3.3.2 Oracle Database

Oracle Database is a powerful relational database management system used for storing and managing structured data. It provides high performance, reliability, and security for handling large volumes of data.

In the Student Course Registration System, Oracle Database is used to store data such as student details, course information, and course registration records. It supports SQL for data manipulation and ensures data integrity through constraints like primary keys and data types.

3.4 Tools & IDEs

3.4.1 Visual Studio Code

Visual Studio Code (VS Code) is used as the development environment for the Student Course Registration System. It is a lightweight code editor that supports Java programming through extensions. VS Code provides features such as syntax

highlighting, code completion, and debugging support, which help improve coding efficiency. It also supports version control integration and extensions for database connectivity. It's simple and user-friendly interface makes it suitable for developing and testing the application.

4. DATABASE DESIGN

4.1 Database Tables

The Student Course Registration System uses a relational database to store student, course, and registration information in a structured manner. The database consists of two main tables: **Student, Course** and **Registration**.

- The **Student** table stores details of students such as student ID, name, and contact information.
- The **Course** table stores information about courses including course ID, course name, credits, and section details.
- The **Registration** table stores the relationship between students and courses, indicating which student is registered for which course.

This separation of data helps in reducing redundancy and improves data organization.

4.2 Table Structure

4.2.1 Student Table

Field Name	Data Type	Description	Constraint
student_id	NUMBER	Student ID	Primary Key
name	VARCHAR2(50)	Student Name	Not Null
branch	VARCHAR2(50)	Branch	Not Null
section	VARCHAR2(10)	Section	Not Null

Table 1: Student Relational Database Table

4.2.2 Course Table:

Field Name	Data Type	Description	Constraint
course_id	NUMBER	Course ID	Primary Key
course_name	VARCHAR2(50)	Course Name	Not Null
credits	NUMBER	Course Credits	Not Null

Table 2: Course Relational Database Table

4.2.3Registration Table:

Field Name	Data Type	Description	Constraint
registration_id	NUMBER	Registration ID	Primary Key
Student_id	NUMBER	Student Reference	Foreign Key(Student Table)
Student_id	NUMBER	Course Reference	Foreign Table(Course Table)
Semester	NUMBER	Semester Number	Not Null

Table 3: Registration Relational Database Table

4.3 Keys and Constraints

Keys and constraints are used to maintain data integrity and ensure accurate data storage.

- **Primary Keys:**
  - Student_id in student table
  - Course_id in course table
  - Registration_id in Registration table
- **Foreign Key:**
  - Student_id in the Registration table references student_id in the student table
  - course_id in Registration table references course_id in Course table
- **NOT NULL Constraint:**
  - Important fields such as student_id, name, branch,course_name, credits and semester cannot be left empty
- **Data Type Constraints:**
  - Each field accepts only valid data types to ensure correctness

These constraints help in maintaining consistency and avoiding duplicate or invalid data.

4.4 Table Relationships

The Student, Course and Registration tables are related through foreign keys.

- Each student can register for multiple courses

- Each course can be taken by multiple students
- The Registration table acts as a bridge between Student and Course tables This represents a **many-to-many relationship** between Student and Course tables, implemented using the Registration table.

5. SYSTEM DESIGN

5.1 System Architecture

The Student Course Registration System follows a **three-layer architecture**:

1. **Presentation Layer (User Interface)**
   - Developed using Java Swing
   - Allows users to interact with the system
2. **Application Layer (Business Logic)**
   - Processes user inputs
   - Performs operations like add, update, delete, search
3. **Data Layer (Database)**
   - Stores student, course, and registration data
   - Managed using Oracle Database

This architecture ensures better organization, security, and maintainability.

5.2 Modules Description

The system is divided into the following modules:

1. Student Management Module

- Adds new student details
- Updates existing student records
- Deletes student data
- Displays student information in table format

2. Course Management Module

- Adds new course details
- Assigns employees to departments
- Deletes course data
- Displays course information

3. Course Registration Module

- Registers students for courses
- Maintains relationship between student and course
- Prevents duplicate registrations

4. Student Subject Registration Count Module

- Displays number of courses registered by each student

- Helps in tracking student course load

5. Section Registration Count Module

- Displays number of students in each section • Helps in analyzing section-wise distribution

6. SYSTEM IMPLEMENTATION

Login Screen:

Displays the login interface where the user enters username and password.

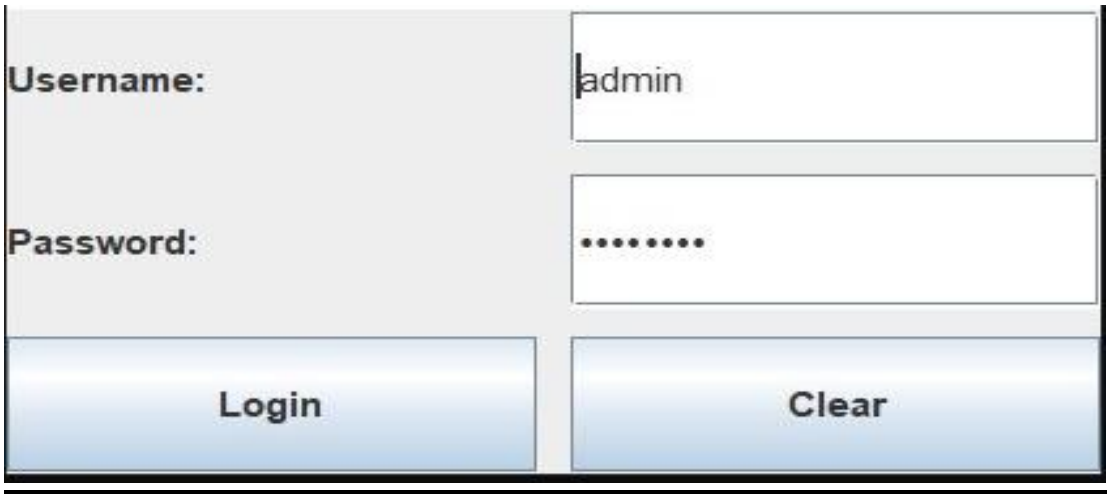

Figure 1: Login Screen of Employee Management System

Main Menu Screen:

Provides options such as Add, Update, Delete, Search, and Display Students.

Figure 2: Main Menu Screen of Student Course Registration System

Student Management Screen:

Shows the form used to enter and manage student details.

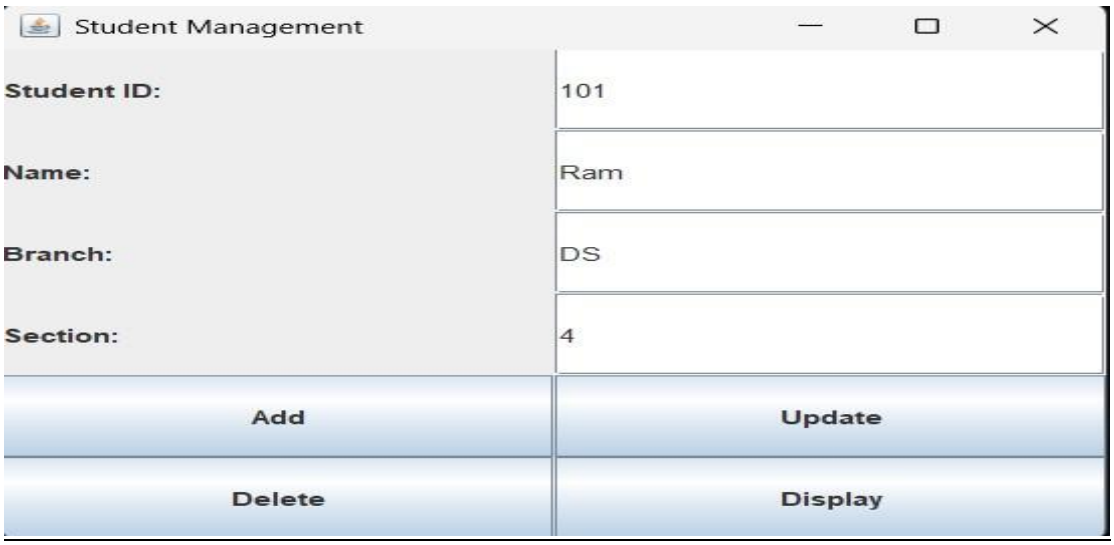

Figure 3: Student Management Screen of Student Course Registration System

Course Management Screen:

Shows the form used to enter and manage course details.

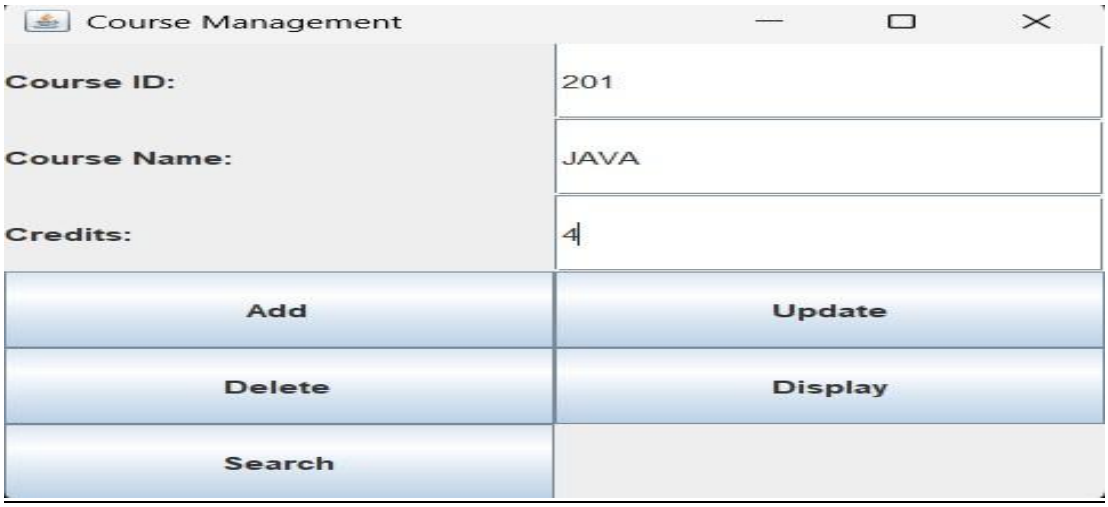

Figure 4: Course Management Screen of Student Course Registration System

Course Registration Screen

Shows the form used to register students for courses.

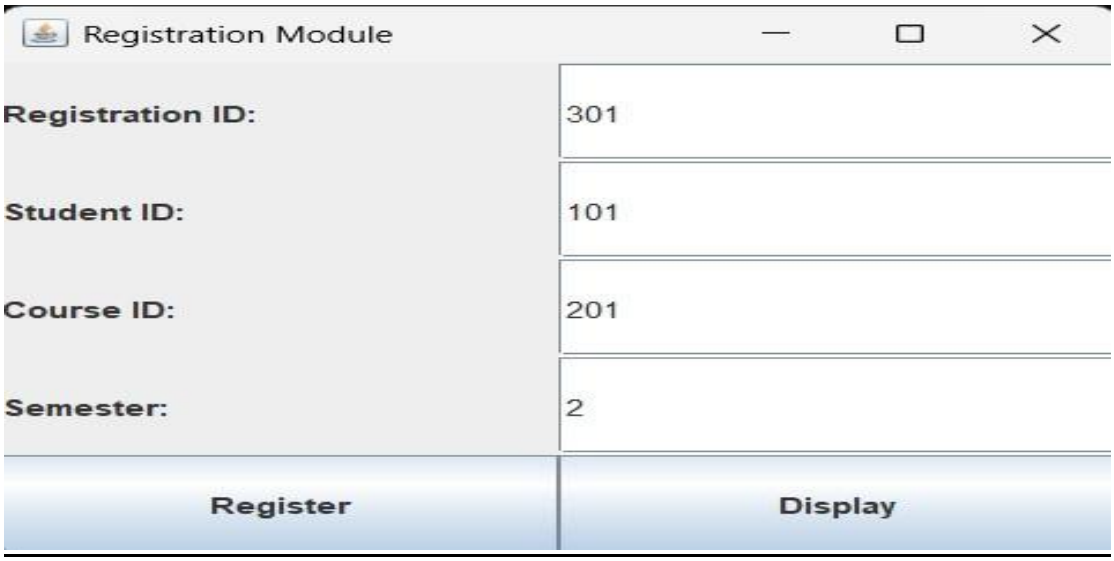

Figure 5: Course Registration Screen of Student Course Registration System

Student Subject Registration Count Screen

Shows the number of courses registered by each student.

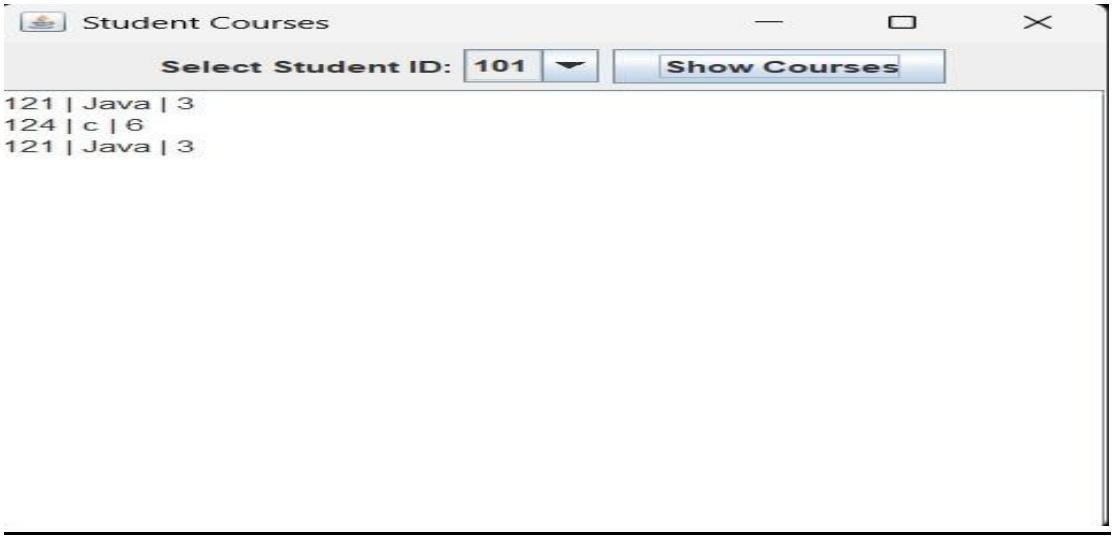

Figure 6: Student Subject Registration Count Screen of Student Course Registration System

Section Registration Count Screen

Shows the number of students in each section registered for particular course.

Figure 7: Student Registration Count Screen of Student Course Registration System

7. CONCLUSION

The Student Course Registration System has been successfully developed to manage student and course registration information in an efficient and organized manner. The system replaces manual registration processes with a computerized solution, thereby reducing errors, saving time, and improving data accuracy.

The application provides essential functionalities such as such as managing student details, course information, and course registration through a user-friendly interface. By using technologies like Java, Swing, JDBC, and Oracle Database, the system ensures reliable performance, secure data handling, and easy access to academic information. Overall, the system improves the efficiency of managing employee data and reduces the workload of administrative staff. It also provides a scalable solution that can be further enhanced in the future by adding features such as web-based access, role-based authentication, and report generation.

In conclusion, the Student Course Registration System meets the project objectives and serves as an effective tool for managing academic registration processes in educational institutions.

8. FUTURE ENHANCEMENTS

The Student Course Registration System developed in this project provides basic functionalities for managing student, course, and registration data. However, the system can be further enhanced by adding advanced features to improve its functionality and usability.

Some possible future enhancements include:

- **Online Registration System:**
  Allowing students to register for courses through a web-based interface from anywhere
- **Role-Based Access Control:**
  Providing different access levels for administrators, faculty, and students
- **Timetable Management:**
  Integrating class scheduling and timetable generation features
- **Advanced Reporting:**
  Generating detailed reports such as student performance, course-wise analysis, and registration statistics
- **Improved User Interface:**
  Enhancing the design for better user experience

These enhancements can make the system more scalable, secure, and suitable for real time academic environments.

9. REFERENCES

The following references were used during the development of the Student Course Registration System:

1. **Y. Daniel Liang,** *Introduction to Java Programming and Data Structures*, 12th Edition, Pearson Education, 2020.
2. **Paul J. Deitel and Harvey Deitel,** *Java: How to Program*, 11th Edition, Pearson, 2018.
3. **Oracle Documentation**, https://docs.oracle.com
4. **Java Documentation (Oracle)**, https://docs.oracle.com/javase
5. **JDBC Tutorial – GeeksforGeeks**, https://www.geeksforgeeks.org/jdbc/
6. **Java Swing Tutorial – TutorialsPoint**, https://www.tutorialspoint.com/java_swing
7. **w3Schools Java Tutorial,** https://www.w3schools.com/java/

APPENDIX

A.1 Sample Code

Database Connection Code:

The following code establishes a connection between the Java application and the Oracle database using JDBC.

DBConnection.java:

```
import    java.sql.*;    public
class DBConnection {
   public    static    Connection    getConnection()    {
Connection con = null;
      try {
         Class.forName("oracle.jdbc.driver.OracleDriver");
con = DriverManager.getConnection(
            "jdbc:oracle:thin:@localhost:1521:xe","system","password");
      } catch (Exception e) {
         System.out.println(e);
      }
return con;
   }
}
```

Insert Student Record:

This code is used to add a new student into the database.

InsertStudent.java:

```
import java.sql.*;

public class InsertStudent {
   public  static  void  main(String[]  args)  {
try {
         Connection con = DBConnection.getConnection();
         String query = "INSERT INTO student VALUES(?,?,?,?)";
         PreparedStatement pst = con.prepareStatement(query);
```

```
            pst.setInt(1, 101);
        pst.setString(2, "Ravi");
        pst.setString(3, "CSE");
        pst.setString(4, "A");

            int  i  =  pst.executeUpdate();
        if (i > 0)
                System.out.println("Student Added Successfully");

            con.close();
          } catch (Exception e) {
            System.out.println(e);
          }
      }
        }
```

Insert Course Record:

This code is used to add a new course into the database.

InsertCourse.java:

```
        import java.sql.*;

        public class InsertCourse {     public
        static void main(String[] args) {        try
        {
            Connection con = DBConnection.getConnection();
            String query = "INSERT INTO course VALUES(?,?,?)";
            PreparedStatement pst = con.prepareStatement(query);

            pst.setInt(1, 201);
            pst.setString(2,          "Data          Structures");
        pst.setInt(3, 4);

            int  i  =  pst.executeUpdate();
        if (i > 0)
```

```
                System.out.println("Course Added Successfully");

            con.close();
        } catch (Exception e) {
            System.out.println(e);
        }
    }
}
```

Insert Registration Record:

This code is used to add a new register of a student for a course.

InsertRegistration.java:

```
import java.sql.*;

public class InsertRegistration {    public
static void main(String[] args) {        try
{
        Connection con = DBConnection.getConnection();
        String query = "INSERT INTO registration VALUES(?,?,?,?)";
        PreparedStatement pst = con.prepareStatement(query);

        pst.setInt(1, 1);
pst.setInt(2, 101);          pst.setInt(3,
201);          pst.setString(4,
"Semester-1");

        int i = pst.executeUpdate();
        if (i > 0)
            System.out.println("Registration Successful");

        con.close();
    } catch (Exception e) {
        System.out.println(e);
    }
```

```
    }
}
```

A.2 SQL Queries

Create Student Table: CREATE TABLE student (student_id NUMBER PRIMARY KEY, name VARCHAR2(100) NOT NULL, branch VARCHAR2(50), section VARCHAR2(10)
);

Create Course Table:

```
CREATE TABLE course (     course_id
NUMBER PRIMARY KEY,     course_name
VARCHAR2(100) NOT NULL,     credits
NUMBER
);
```

Create Registration Table:

```
CREATE TABLE registration (
registration_id NUMBER PRIMARY KEY,
student_id NUMBER,    course_id NUMBER,
semester VARCHAR2(20),
   FOREIGN KEY (student_id) REFERENCES student(student_id),
   FOREIGN KEY (course_id) REFERENCES course(course_id)
);
```